1. ТЕОРЕТИЧНІ ОСНОВИ

Звичайні функції завжди повертають одне значення. Але існують й інші задачі, коли потрібно повернути з функції декілька значень або зовсім не повертати нічого (наприклад, функція виведення вмісту двовимірного масиву у вигляді таблиці). У такому раз функція може оформлюватися як функція з типом **void**.

Опис **void**-функції нічим особливим не відрізняється від опису функції, що повертає значення. По суті, існують тільки такі відмінності:

- у заголовку вказується службове слово **void** як тип функції;
- оператор **return** не включається в текст функції або, у разі його наявності, після службового слова **return** не вказується ніякий вираз, за яким повинно обчислюватися значення, що повертається, оскільки ніяке таке значення не існує.

Якщо виклик функції, що повертає значення, може з'являтися в будь-якому виразі, то цього не можна сказати про **void**-функцій. Виклик таких функцій здійснюється у вигляді окремого оператора.

Важливим моментом при використанні **void**-функцій є застосування покажчиків як їх формальних параметрів.

При звертанні до функцій за умовчанням, тобто у всіх випадках, коли не зазначено інший спосіб (а саме, невикористані посилання[1]) здійснюється так звана **передача фактичних параметрів по значенню** – виконується обчислення значень фактичних параметрів, після чого вони записуються у відповідні формальні параметри. Як тільки починається виконання функції, жодні зміни значення формального параметра не впливають на значення відповідного фактичного параметра. Всередині тіла функції здійснюється обробка формальних параметрів, причому по закінченні роботи функції ці параметри знищуються. Тому через формальні параметри передача значень з тіла функції в точку виклику неможлива.

У мові Сі дозволена робота з адресами змінних. Для цього визначено спеціальний тип даних – покажчик. Оголошення змінних, що мають такий тип здійснюється зазначенням так званого базового типу, перед яким указується символ `*`.

Так, оголошені нижче змінні є відповідно звичайною змінною типу **double** й покажчиком на тип **double**:

```
double d;
```

[1] При оголошенні посилання використовують конструкція, що має такий вигляд: `тип &`.

```
double *ptD;
```

Змінні, що використовуються в програмі, мають свої адреси. Для отримання адреси змінної використовується операція `&`. Наприклад, при наявності наведених вище оголошень ім'я `d` позначає в програмі змінну, а вираз `&d` – її адресу.

Адреса змінної може бути записана в іншу змінну, але ця змінна повинна мати тип «покажчик», причому базовий тип цього покажчика **повинен співпадати з типом змінної**; наприклад

```
ptD = &d;
```

Присвоювання адреси змінної покажчику базового типу заборонено. Наприклад, для оператора

```
int *ptI = &d;
```

компілятор генеруватиме помилку.

Винятком є покажчик на тип void. Такий покажчик приймає покажчики будь-якого базового типу. Наприклад, оператор

```
void *ptV = &d;
```

є цілком коректним.

Якщо до покажчика застосувати унарну операцію * (операція розіменування), то забезпечується доступ до змінної, адреса якої записана в покажчик. Наприклад:

```
*ptD = -3.1415;     // Значення -3.1415 запишеться в d
cout << *ptD;       // Доступ до d через покажчик
```

Літерали й вирази не мають адрес, у зв'язку із чим отримати покажчики на них неможливо.

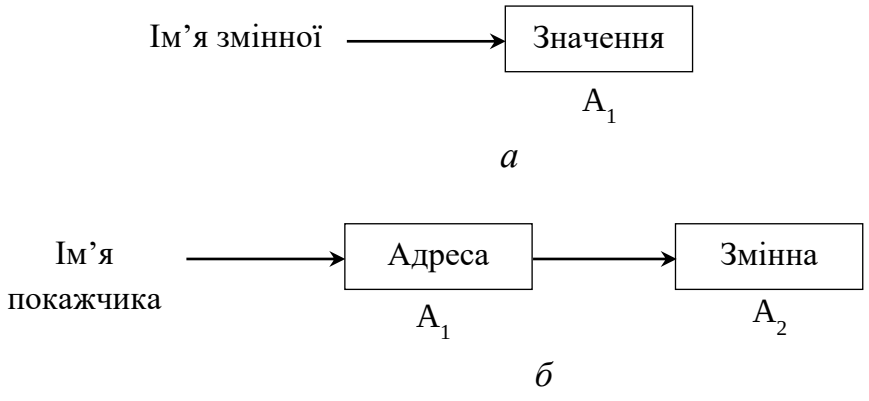


Рис. 1.1. Взаємодія покажчика і звичайної змінної:
а – звичайна змінна; *б* – взаємодія з покажчиком

Використанням покажчиків як параметрів функцій забезпечується опосередкована можливість змінювати об'єкти, що знаходяться поза функцією діями функції, що була викликана. Для цього у функцію за допомогою покажчика передається адреса об'єкта (змінної), який визначений у тій функції, що забезпечила виклик. Всередині ж функції за допомогою розіменування покажчика забезпечується доступ до цього об'єкта.

Приклад запису прототипу **void**-функції з параметрами-покажчиками:

```
void FindMaxMin(int x, int y, int *max, int *min);
```

Треба пам'ятати, що значенням покажчика можуть бути тільки об'єкти, що мають свою адресу в пам'яті. Тому замість формального параметра-покажчика можна підставляти тільки вираз, результатом якого є покажчик, базовий тип якого той самий, що й базовий тип формального параметра.

Наголосимо: звичайні функції, що повертають значення, також можуть мати параметри-покажчики.

2. ПРИКЛАДИ ПРОГРАМ

Приклад 1. Дано дві дійсні змінні. Здійснити взаємний обмін їх значень. Розробити функцію обміну значеннями двох дійсних змінних.

Розв'язок.

```
#include <iostream>
#include <conio.h>
using namespace std;

void SwapAB(double *a, double * b);          // Формальні
                                        // параметри-покажчики
int main()
{
    double a, b;
    cout << "a = ";
    cin >> a;
    cout << "b = ";
    cin >> b;
    SwapAB(&a, &b); //Фактичні параметри - адреси змінних
    cout << "a = " << a << "   b = " << b << '\n';
    cout << "Press any key";
    _getch();
```

```
    return 0;
}

void SwapAB(double *a, double *b)
{
    double buf = *a;
    *a = *b;
    *b = buf;
    return;
}
```

Приклад 2. Написати програму, що повідомляє, якою кількістю монет номіналом 1, 2 і 25 коп. можна видати решту, величина якої знаходиться в інтервалі від 1 до 99 коп. Кількість монет повинна бути мінімальною. Інші монети не використовуються.

Розв'язок.

```
#include <iostream>
#include <conio.h>

using namespace std;

void Coin(int *N_1, int *N_2, int *N_25, int n); // Перші
                    // три формальні параметри – покажчики
int main()
{
  int n, Count_1, Count_2, Count_25;
  cout << "n = ";
  cin >> n;
          // Нижче при виклику функції перші три фактичні
                  // фактичні параметри – адреси змінних,
                           // а останній – звичайна змінна
  Coin(&Count_1, &Count_2, &Count_25, n);
  cout << n << " kop. == " << Count_25 << " * 25 kop. + "
                       << Count_2 << " * 2 kop. + "
                       << Count_1 << " * 1 kop. " << '\n';
  cout << "Press any key";
  _getch();
  return 0;
}
void Coin(int *N_1, int *N_2, int *N_25, int n)
```

```
{          // Нижче здійснюється розіменування покажчиків
    *N_25 = n / 25;
    *N_2 = (n - *N_25 * 25) / 2;
    *N_1 = n - *N_25 * 25 -  *N_2 * 2;
    return;
}
```

Приклад 3. Дано натуральне числа *n* і двовимірний масив дійсних чисел розміром $2 \times n$, за допомогою якого задано координати *n* точок на площині. Визначити номери двох найвіддаленіших точок і відстань між ними.

Розв'язок.

```
#include <iostream>
#include <conio.h>

using namespace std;
                                        // Прототипи функцій
void ReadPoints(double Points[2][100], int n);
void MostDistantPoints(int *n1, int *n2, double *Len,
       double W[2][100], int n);      // *n1, *n2 – номери
       // найвіддаленіших точок; *Len – відстань між ними
int main()
{
    double W[2][100];
    double X1, Y1, X2, Y2, Len;
    int n, n1, n2;
    cout << "n = ";
    cin >> n;
    ReadPoints(W, n);
    MostDistantPoints(&n1, &n2, &Len, W, n);
    cout << "X1 == " << W[0][n1] <<
            "\tY1 == " << W[1][n1]  << "\n";
    cout << "X2 == " << W[0][n2] <<
            "\tY2 == " << W[1][n2]  << "\n";
    cout << "Length == " << Len  << "\n";
    cout << "Press any key";
    _getch();
    return 0;              // Нормальне завершення функції
}
                                    // Функція введення масиву
```

```
void ReadPoints(double Points[2][100], int n)
{            // Ім'я масиву - покажчик; Його індексування
    for (int j = 0; j < n; j++)             // забезпечує
    {                  // доступ до елементів масиву зовні
        cout << "X[" << j + 1 << "] = ";       // функції
        cin >> Points[0][j];              // і їх змінення
        cout << "Y[" << j + 1 << "] = ";
        cin >> Points[1][j];
    }
    return;                              // Вихід з void-функції
}
                                  // Функція виведення масиву
void MostDistantPoints(int *n1, int *n2, double *Len,
                       double W[2][100], int n)
{          // Нижче здійснюється розіменування покажчиків
    *n1 = 0;
    *n2 = 0;
    double L, M = 0;
    for (int i = 0; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            if ((L = (W[0][i] - W[0][j]) *
                (W[0][i] - W[0][j]) +
                (W[1][i] - W[1][j]) *
                (W[1][i] - W[1][j])) > M)
            {
                *n1 = i;
                *n2 = j;
                M = L;
            }
    *Len = sqrt(M);
    return;                              // Вихід з void-функції
}
```

Приклад 4. Дано натуральне число n ($n \le 20$) і дійсні числа x_1, x_2, …, x_n. Замінити елементи послідовності x_1, x_2, …, x_n новими значеннями $\tilde{x}_1$, $\tilde{x}_2$, …, $\tilde{x}_n$, обчислюваними за таким правилом: $\tilde{x}_i = (x_i - \bar{x})/s$, де $\bar{x} = \dfrac{\sum_{i=1}^{n} x_i}{n}$,

$s^2 = \dfrac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n-1}$, $i = 1, 2, \ldots, n$. Визначити три функції – уведення послідовності, обчислення значень $\bar{x}$ (середнє) і s^2 (оцінка величини, що має назву «дисперсія») і перетворення послідовності.

Розв'язок.

```
#include <iostream>
#include <conio.h>
using namespace std;
                                        // Прототипи функцій
int ReadArray(double *x, int *n, int N = 20); // Уведення
void PrintArray(double *x, int n);    // Виведення масиву
double Sum(double *x, int n); // Підсумовування елементів
void MeanAndDisp(double *Mean, double *Variance,// Обчис-
     double *x, int n);   // лення середнього і дисперсії
int Transformtion(double *x, int n,       // Перетворення
               double m, double s);              // масиву

int main()
{
    double x[20], m, s;
    int n;
    do
    {
    } while (ReadArray(x, &n,
                       sizeof(x) / sizeof(x[0])) == -1);
    MeanAndDisp(&m, &s, x, n);
    if (Transformtion(x, n, m, s) == -1)
        cout << "Error\n";
    else
        PrintArray(x, n);
    cout << "Press any key";
    _getch();
    return 0;
}
                                    // Функція введення масиву
int ReadArray(double *x, int *n, int N)
{
```

```
    cout << "n = ";
    cin >> *n;                  // Розіменування покажчика
    if (*n < 1 || *n > sizeof(x))   // Невірне значення n
        return -1;              // Код помилкової ситуації
    for (int i = 0; i < *n; i++)
    {           // Оскільки ім'я масиву – покажчик, можна
        cout << "x[" << i << "] = ";   // використовувати
        cin >> x[i];                   // індексування
    }
    return 0;              // Нормальне завершення функції
}
                              // Функція виведення масиву
void PrintArray(double *x, int n)
{
    for (int i = 0; i < n; i ++)
        cout << x[i] << "  ";
    cout << '\n';
    return;                         // Вихід з void-функції
}
                // Функція підсумовування елементів масиву
double Sum(double *x, int n)
{
    double m = 0;
    for (int i = 0; i < n; i++)
        m += x[i];
    return m;
}
      // Функція обчислення середнього і оцінки дисперсії
void MeanAndDisp(double *Mean, double *Variance,
              double *x, int n)
{   // Оскільки Mean і Variance - покажчики, їх розімену-
   // вання забезпечує доступ до відповідних змінних поза
    *Mean = Sum(x, n) / n;                  // функцією
    *Variance = 0;
    for (int i = 0; i < n; i ++)
        *Variance += (x[i] - *Mean) *(x[i] - *Mean);
    *Variance /= (n - 1);
    return;                         // Вихід з void-функції
}
```

```
                                    // Функція перетворення масиву
int Transformtion(double *x, int n, double m, double s)
{  // Оскільки x – покажчик, уміст масиву змінюватиметься
    if (n == 1)                     // Перетворення неможливе?
        return -1;                  // Код помилкової ситуації
    for (int i = 0; i < n; i++)     // Перетворення масиву
        x[i] = (x[i] - m) / sqrt(s);
    return 0;                   // Нормальне завершення функції
}
```

Приклад 5. Дано натуральне число n ($n \le 100$) і два цілочислових масиви V і W, кожен з яких має не більше n елементів. Видалити з масиву V усі елементи, що містяться в масиві W. Визначити і використати функцію, що видаляє з одновимірного цілочислового масиву всі числа, які входять у другий масив.

Розв'язок.

```
#include <iostream>
#include <conio.h>
using namespace std;
                                            // Прототипи функцій
int ReadArray(int *x, int *n, int N = 100);
void PrintArray(int *x, int n);
void Difference(int *V, int *nV, int *W, int nW);

int main()
{
    int V[100], W[100];
    int nV, nW;
    cout << "Array V:\n";
    do          // Уводимо розмір масиву V і його елементи
    {
    } while (ReadArray(V, &nV,
                       sizeof(V) / sizeof(V[0])) == -1);
    cout << "Array W:\n";
    do          // Уводимо розмір масиву W і його елементи
    {
    } while (ReadArray(W, &nW,
                       sizeof(W) / sizeof(W[0])) == -1);
    Difference(V, &nV, W, nW);     // Обчислюємо V = V – W
```

```
    PrintArray(V, nV);
    cout << "Press any key";
    _getch();
    return 0;
}
                                    // Функція введення масиву
int ReadArray(int *x, int *n, int N)
{
    cout << "Count of elements: ";
    cin >> *n;                      // Розіменування покажчика
    if (*n < 1 || *n > N)             // Невірне значення n
        return -1;                  // Код помилкової ситуації
    for (int i = 0; i < *n; i++)
    {     // Оскільки ім'я масиву – покажчик, можна вико-
        cout << "Element[" << i << "] = "; // ристовувати
        cin >> x[i];                        // індексування
    }
    return 0;                  // Нормальне завершення функції
}
                                   // Функція виведення масиву
void PrintArray(int *x, int n)
{
    if (n == 0)
        cout << "Empty";
    else
        for (int i = 0; i < n; i++)
            cout << x[i] << "  ";
    cout << '\n';
    return;                             // Вихід з void-функції
}
                // Функція отримання нового вмісту масиву V
void Difference(int *V, int *nV, int *W, int nW)
{  // Оскільки x – покажчик, уміст масиву змінюватиметься
    bool Flag;
    int i = 0, k = 0;
    while (i < *nV)
    {
        Flag = true;     // Вважаємо, що V[i] відсутнє в W
        for (int j = 0; j < nW; j++)
```

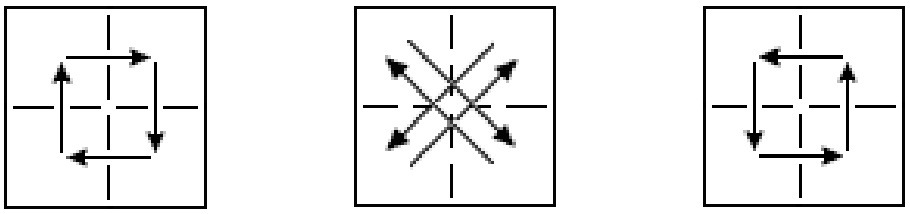
```
            if (V[i] == W[j])
            {       // Значення V[i] міститься в масиві W
                Flag = false;
                break;
            }
        if (Flag)       // Якщо  V[i] відсутнє в масиві W,
        {           // записуємо його в позицію k масиву V
            V[k] = V[i];
            k++;
        }
        i++;     // Переходимо до розгляду нового елемента
    }
    *nV = k;                          // Новий розмір масиву V
    return;                           // Вихід з void-функції
}
```

3. ВАРІАНТИ ЗАДАЧ

1. Дано два натуральні числа m і n. Знайти їх найбільший спільний дільник і найменше спільне кратне. Визначити в програмі власну функцію обчислення цих двох величин.
2. Дано цілі числа $p, n_0, d_0, n_1, d_1, \cdots, n_p, d_p, a, b$ $(d_0 \cdot d_1 \cdot \ldots \cdot d_p \cdot b \neq 0,\ p \geq 0)$. Обчислити $\frac{n_p}{d_p}\left(\frac{a}{b}\right)^p + \frac{n_{p-1}}{d_{p-1}}\left(\frac{a}{b}\right)^{p-1} + \ldots + \frac{n_0}{d_0}$, одержавши результат у вигляді простого дробу. Визначити функції повного скорочення числа, заданого чисельником і знаменником, а також функції множення та додавання простих дробів.
3. Дано одновимірний масив натуральних чисел. Видалити в цьому масиві всі прості числа. Описати функції перевірки числа на те, що воно є простим, та видалення з масиву всіх простих чисел. Скористатися цими функціями для розв’язання сформульованої задачі.
4. Описати функцію що переписує вміст масиву з n цілих чисел ($n \leq 100$) у два масиви. В один масив повинні бути записані всі парні числа, а в інший – всі непарні. Порядок слідування парних і непарних чисел повинен бути збереженим. Налаштувати функцію на декількох прикладах.
5. Дано натуральне число n ($n \leq 100$) і цілочисловий масив з n елементів. Знайти довжину найдовшої послідовності відмінних від нуля елементів

масиву, що йдуть підряд. Переписати цю послідовність в інший масив. Розробити функцію формування вказаного масиву.

6. Дано натуральне число n ($n \le 100$) і цілі числа a_1, a_2, …, a_n. Знайти відрізок масиву максимальної довжини, в якому перше число дорівнює останньому, друге – передостанньому і т. д. Визначити функцію одержання довжини і номера першого елемента цього відрізка (якщо їх декілька, то першого з них).
7. Описати функцію, що виконує обмін рядків з номерами k_1 і k_2 у дійсній матриці $\mathbf{A} = (a_{ij})_{mn}$. Використовуючи цю функцію, поміняти місцями в заданій дійсній матриці **A** розміром $m \times n$ рядки, що містять мінімальний і максимальний елементи.
8. Описати функцію, що виконує обмін стовпців з номерами k_1 і k_2 у дійсній матриці $\mathbf{A} = (a_{ij})_{mn}$. Використовуючи цю функцію, поміняти місцями в заданій дійсній матриці **A** розміром $m \times n$ стовпці, що містять найбільшу і найменшу кількість додатних елементів.
9. Описати функцію, що виконує транспонування дійсної квадратної матриці **A** порядку m без формування нової матриці. Використати цю функцію для перетворення заданої квадратної матриці **A** порядку m у транспоновану матрицю. **Примітка.** *Транспонована матриця* формується з первинної записом рядків у вигляді стовпців (або, що те ж саме, стовпців у вигляді рядків).
10. Описати та використати в програмі функцію для блокового переносу частин прямокутного квадратного масиву з парними розмірами відповідно до схем, наведених на рис. 4.1, *а–в*.



а *б* *в*

Рис. 4.1

11. *Сумою* двох матриць **A** і **B** розміром $m \times n$ називається така матриця **C** розміром $m \times n$, елементи якої дорівнюють сумі відповідних елементів матриць-доданків:

$$c_{ij} = a_{ij} + b_{ij},\ i = 1, 2, \ldots, m,\ j = 1, 2, \ldots, n.$$

Описати функцію отримання суми двох матриць і продемонструвати її використання.

12. *Добутком* двох матриць **A** (розміром $m \times n$) і **B** (розміром $n \times p$) називається така матриця **C** (розміром $m \times p$), елементи якої дорівнюють сумі попарних добутків відповідних елементів рядка першої матриці і стовпця другої матриці:

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj},\ i = 1, 2, \ldots,\ m,\ j = 1, 2, \ldots, p.$$

Описати функцію отримання добутку двох матриць. Функція повинна мати параметр, призначений для передавання інформації про можливість або неможливість отримання добутку (наприклад, у цей параметр може бути записане значення –1 у разі неможливості отримання добутку і значення 0 у противному випадку).

13. Описати функцію, що перетворює дійсну матрицю **A** розміром $m \times n$, віднімаючи з рядка i_1 рядок i_2, помножений на задане дійсне число g. За допомогою цієї функції, використовуючи як допоміжний той рядок, який містить максимальний за модулем елемент заданої матриці **A** розміром $m \times n$, замінити нулями в інших її рядках елементи стовпця з максимальним за модулем елементом матриці.

14. Одержати масив усіх простих чисел, що не перевищують задане натуральне число n. Для розв'язання задачі описати функцію, яка ґрунтується на способі, що зветься «Решето Ератосфена». Суть способу в наступному. Нехай дано послідовність натуральних чисел 2, 3, …, n. Перше просте число 2. Викреслимо його і всі кратні йому числа. Перше з чисел, що залишилися, знову є простим, і можна виконати ті ж самі дії над цим числом і т. д.